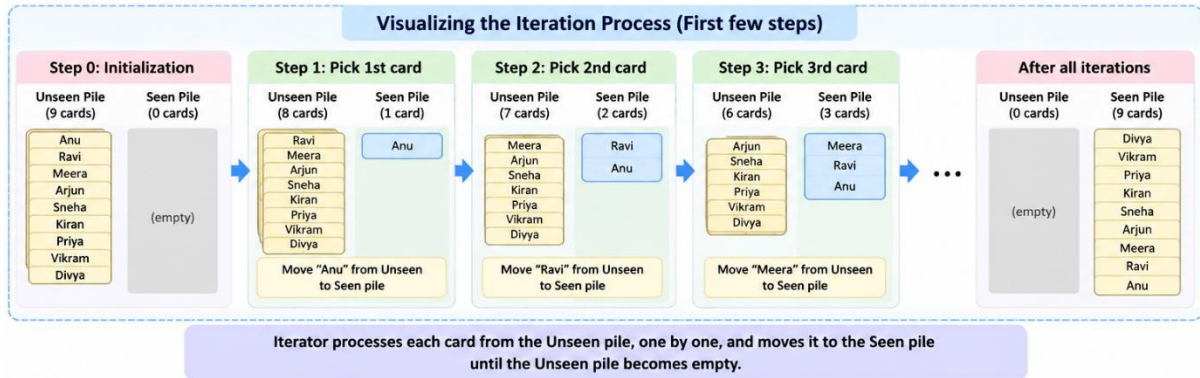
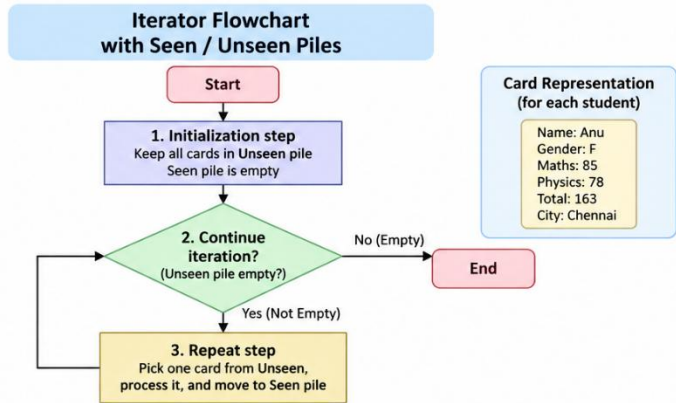
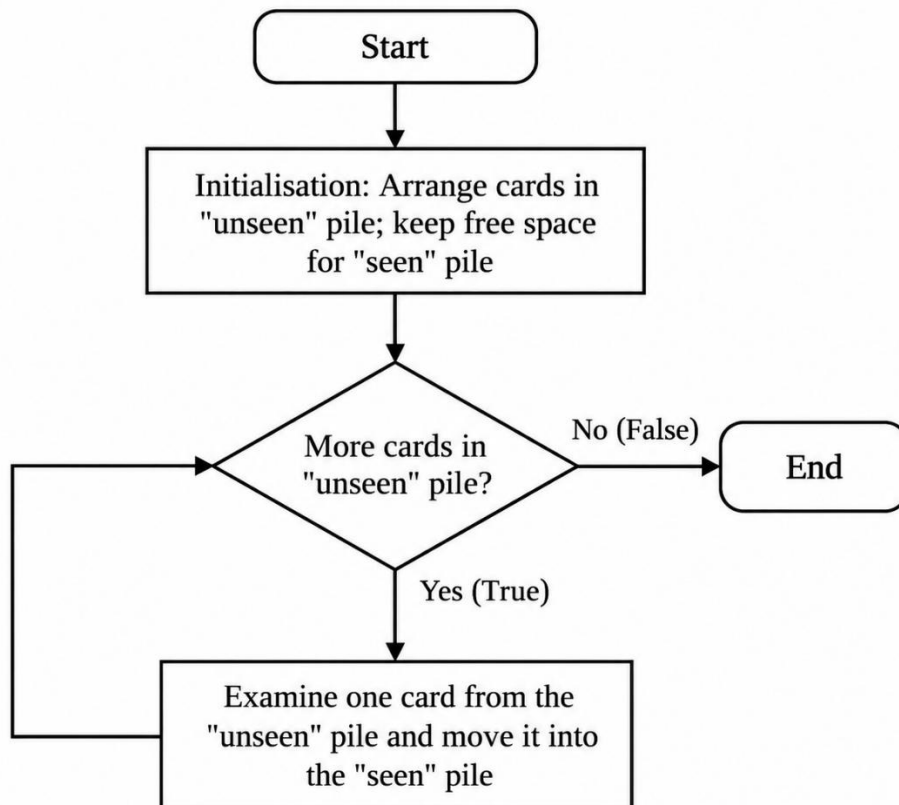
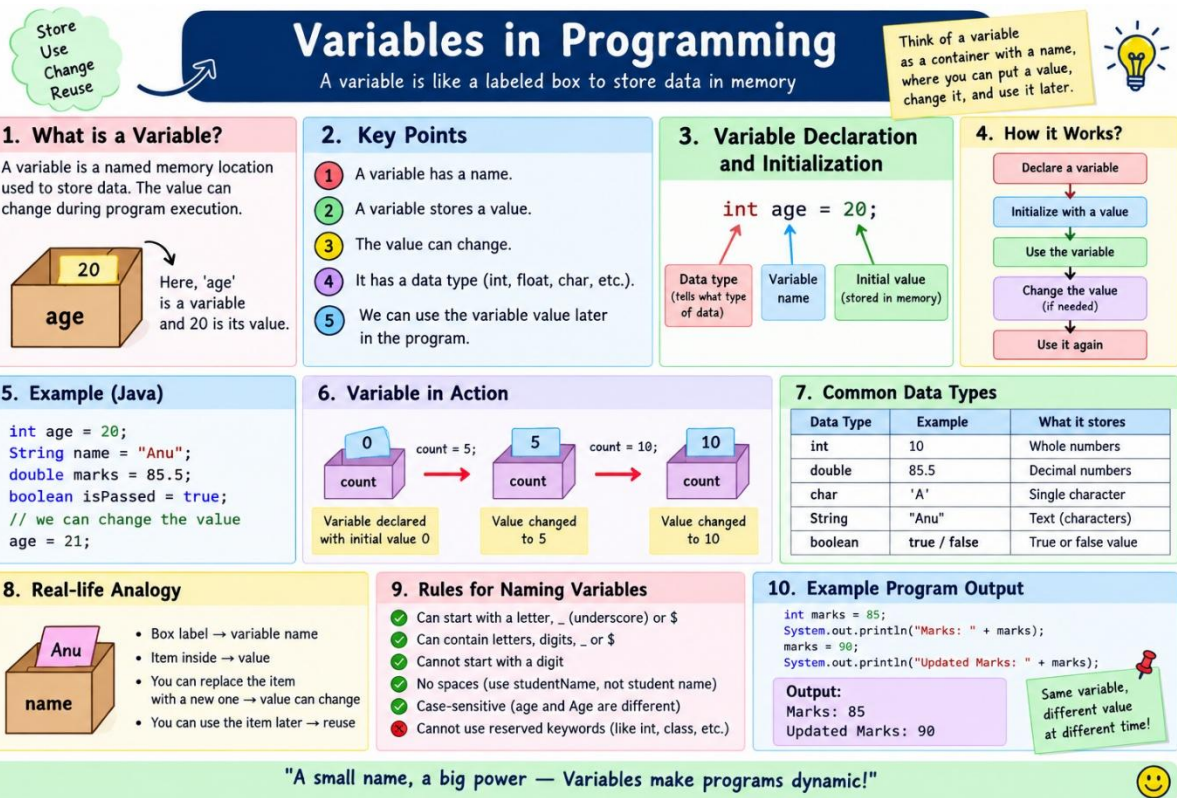


Student Dataset (Cards)					
Name	Gender	Maths	Physics	Total	City
Anu	F	85	78	163	Chennai
Ravi	M	92	88	180	Hyderabad
Meera	F	76	84	160	Bengaluru
Arjun	M	68	73	141	Chennai
Sneha	F	95	91	186	Visakhapatnam
Kiran	M	81	79	160	Vijayawada
Priya	F	89	94	183	Hyderabad
Vikram	M	74	68	142	Bengaluru
Divya	F	98	96	194	Chennai

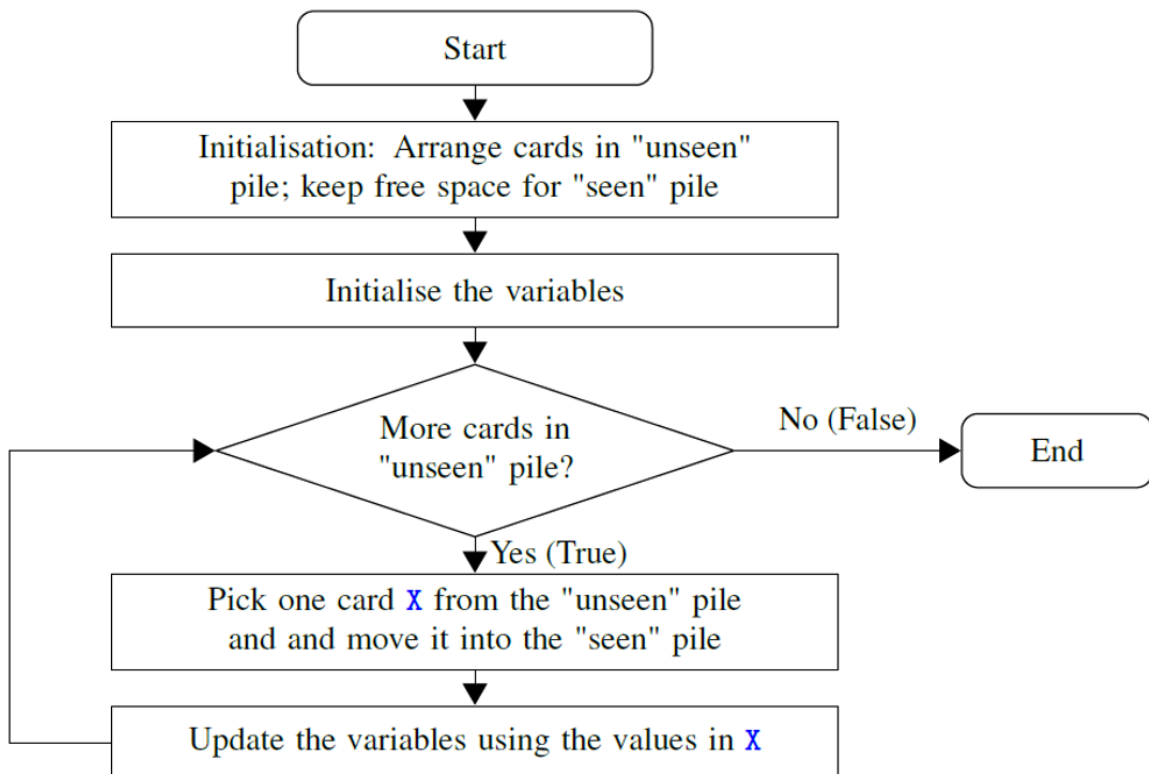


Iterator flowchart for cards



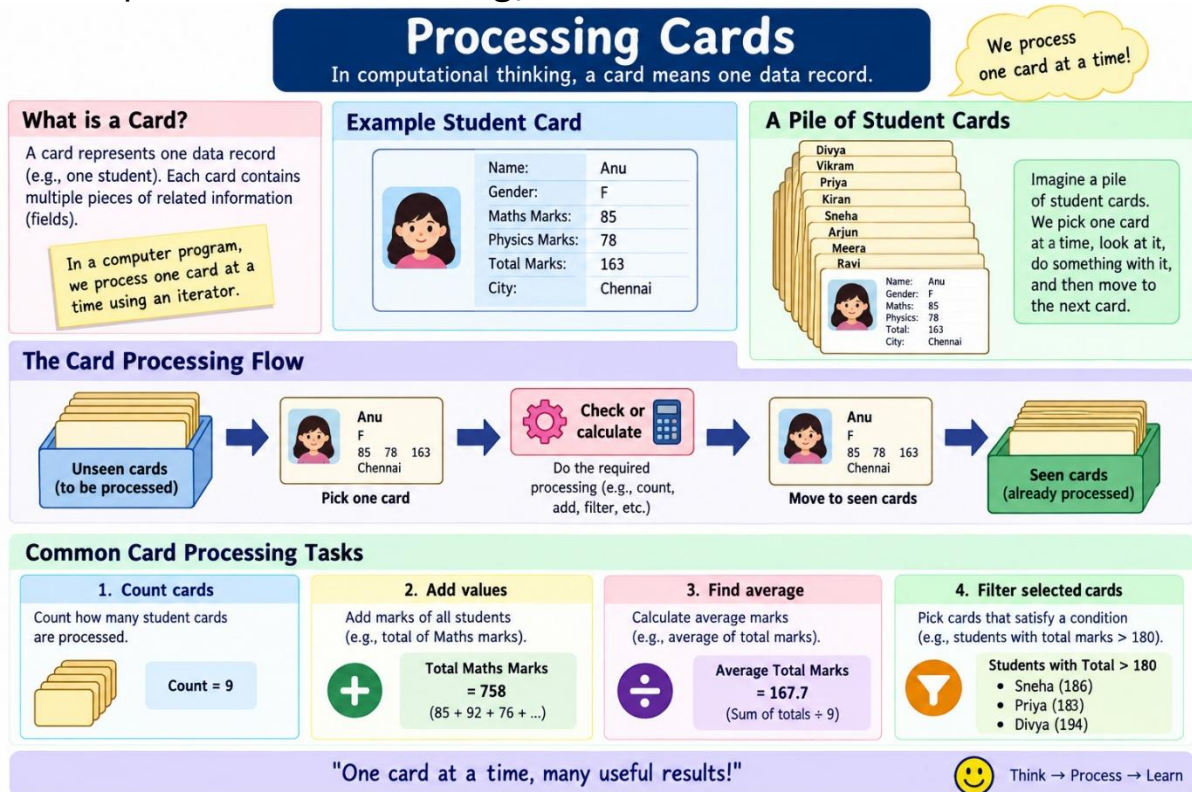


Generic flowchart for iterator with variables



Processing Cards

In computational thinking, a card means one data record.



Example student card:

text

Name: Anu

Gender: F

Maths Marks: 85

Physics Marks: 78

Total Marks: 163

City: Chennai

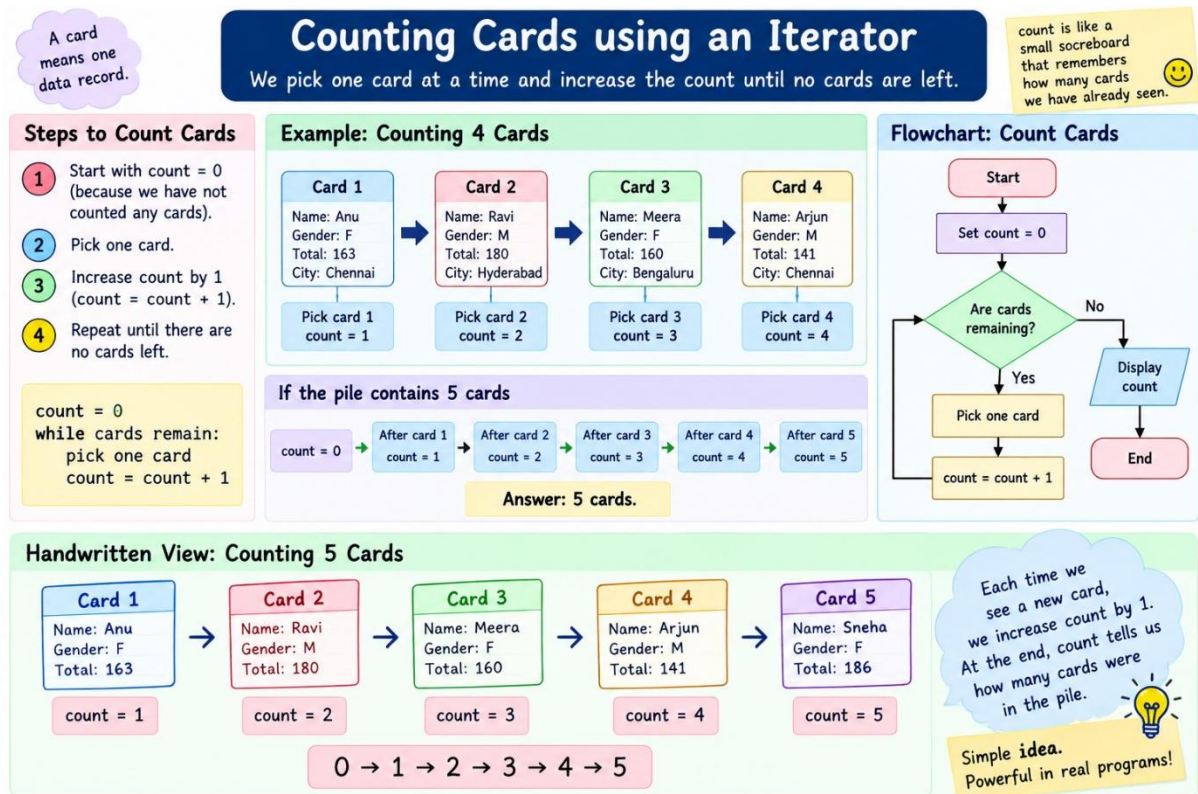
Imagine a pile of student cards. We pick one card at a time, look at it, do something with it, and then move to the next card.

Unseen cards → Pick one card → Check or calculate → Seen cards

Card processing

- Count cards
- Add values
- Find average

Filter selected cards



Imagine a pile of cards. We want to know **how many cards are in the pile.**

We can count them one by one:

1. Start with count = 0 because we have not counted any cards.
2. Pick one card.
3. Increase count by 1.
4. Repeat until there are no cards left.

```
count = 0
while cards remain:
    pick one card
    count = count + 1
```

Start: count = 0

Pick card 1 → count = 1

Pick card 2 → count = 2

Pick card 3 → count = 3

Pick card 4 → count = 4

The variable count acts like a small scoreboard that remembers how many cards we have already seen.

```
Start
↓
Set count = 0
↓
Are cards remaining?
├─ No → Display count → End
└─ Yes
    ↓
    Pick one card
    ↓
    count = count + 1
    ↓
    Check for more cards
```

Simple example

If the pile contains 5 cards:

count = 0

After card 1: count = 1

After card 2: count = 2

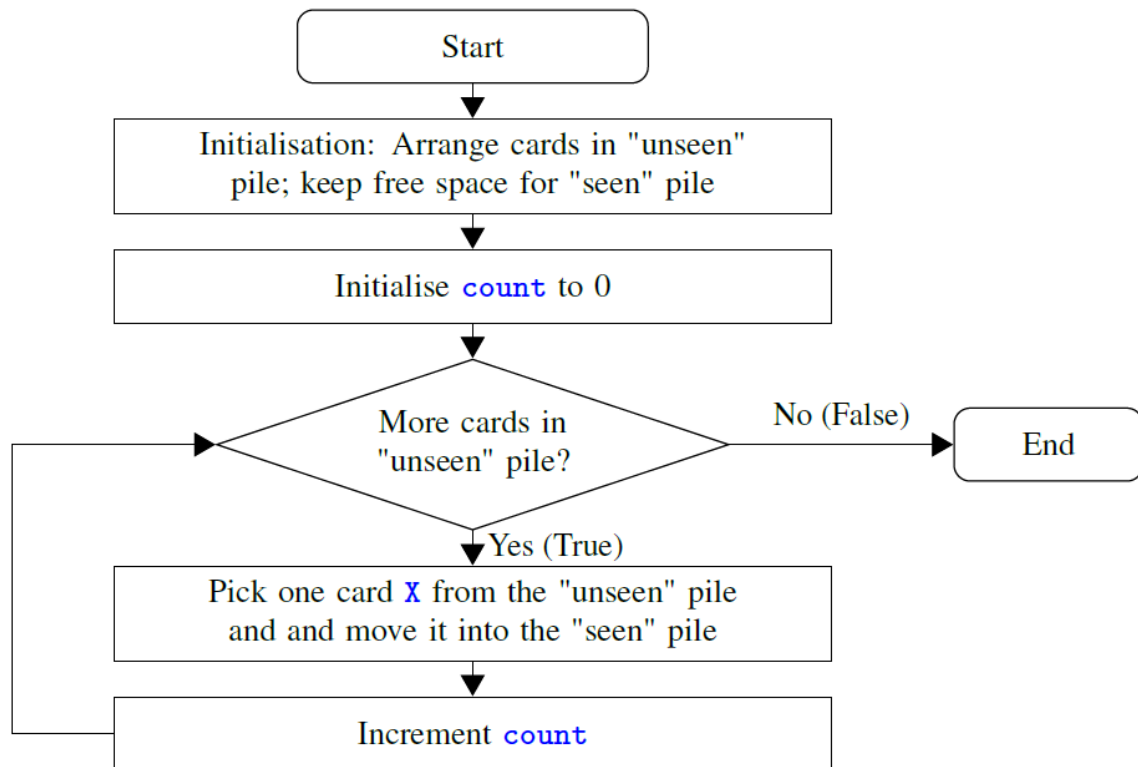
After card 3: count = 3

After card 4: count = 4

After card 5: count = 5

Answer: 5 cards.

0 → 1 → 2 → 3 → 4 → 5



Sum

A **sum** is the total obtained by adding values.

For example:

- Total marks of all students.
- Total marks of girls.
- Total bill amount.
- Total number of letters in selected words.

Sum method

1. Start the sum at 0.
2. Pick one card.
3. Add the required value to the sum.
4. Repeat for every card.

Each card contains information about one student:

Student Maths Physics Total

Anu	85	78	163
Ravi	92	88	180
Meera	76	84	160
Arjun	68	73	141

Counting vs. Adding

When we were **counting cards**, we didn't care what was written on the cards.

We simply did:

```
count = 0
```

Pick a card

```
count = count + 1
```

So:

Card 1 → count = 1

Card 2 → count = 2

Card 3 → count = 3

Card 4 → count = 4

Here, count answers:

"How many cards have I seen?"

Now we want to ADD something

Suppose we want to find the **sum of Maths marks**.

Now we **must look inside each card**.

We need another variable called an **accumulator**.

```
sum = 0
```

Think of sum as a **box that collects marks**.



One card at a time

Start

↓

```
sum = 0
```

↓

Pick Anu → Maths = 85

↓

```
sum = 0 + 85 = 85
```

↓

Pick Ravi → Maths = 92

↓

```
sum = 85 + 92 = 177
```

↓

Pick Meera → Maths = 76

↓

```
sum = 177 + 76 = 253
```

↓

Pick Arjun → Maths = 68

↓

```
sum = 253 + 68 = 321
```

So the final answer is:

Maths total = 321

The important idea

The **count** tells us **how many cards** we processed.

The **sum/accumulator** tells us **how much value we have collected**.

COUNT

SUM / ACCUMULATOR

count = 0

sum = 0

Card 1 → +1

Card 1 → +85

Card 2 → +1

Card 2 → +92

Card 3 → +1

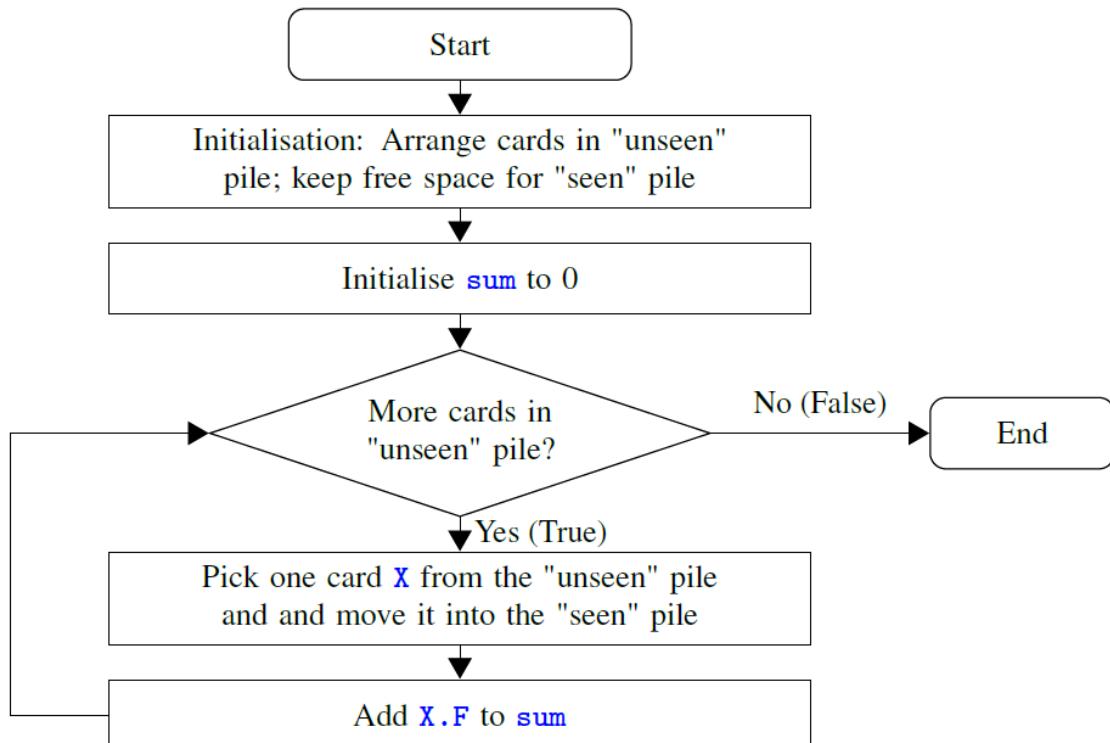
Card 3 → +76

Card 4 → +1

Card 4 → +68

Final count = 4

Final sum = 321



Average

To find the **average** of a field, such as physics marks, bill amount, or letter count, use this formula:

$$\text{Average} = \frac{\text{Total value}}{\text{Number of cards}}$$

Here:

- sum stores the total value of the selected field.
- count stores the number of cards.
- average stores the final result.

Two-pass method

One way is to go through the cards twice:

1. Go through all cards and calculate sum.
2. Go through all cards again and calculate count.
3. Divide sum by count.

This works, but it is inefficient because the cards are examined two times.

One-pass method

A better approach is to calculate sum and count together while going through the cards only once.

For each card:

1. Select a card from the "unseen" pile.
2. Move it to the "seen" pile.
3. Add the required field value to sum.
4. Increase count by 1.
5. Repeat until there are no cards left.
6. Calculate:

$$\text{average} = \frac{\text{sum}}{\text{count}}$$

The flowchart follows this logic:

text

sum = 0

count = 0

while there are cards in the unseen pile:

 choose a card X

 move X to the seen pile

```
sum = sum + X.F
count = count + 1
```

```
average = sum / count
```

Here, X.F means the value of field F on card X. For example, if F is physics_marks, then X.F represents the physics marks on the selected student's card.

Example

Suppose three students have physics marks of 70, 80, and 90.

Card X.F sum count

1 70 70 1

2 80 150 2

3 90 240 3

After all cards are processed:

$$\text{average} = \frac{240}{3} = 80$$

Thus, the average physics mark is 80.

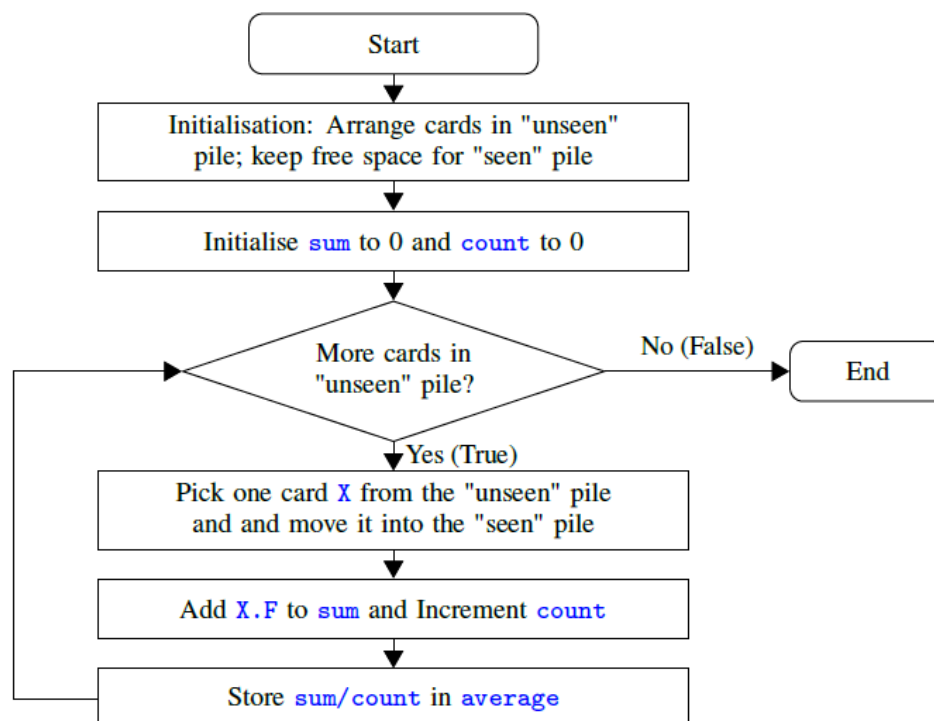


Figure 3.4: Flowchart for finding average

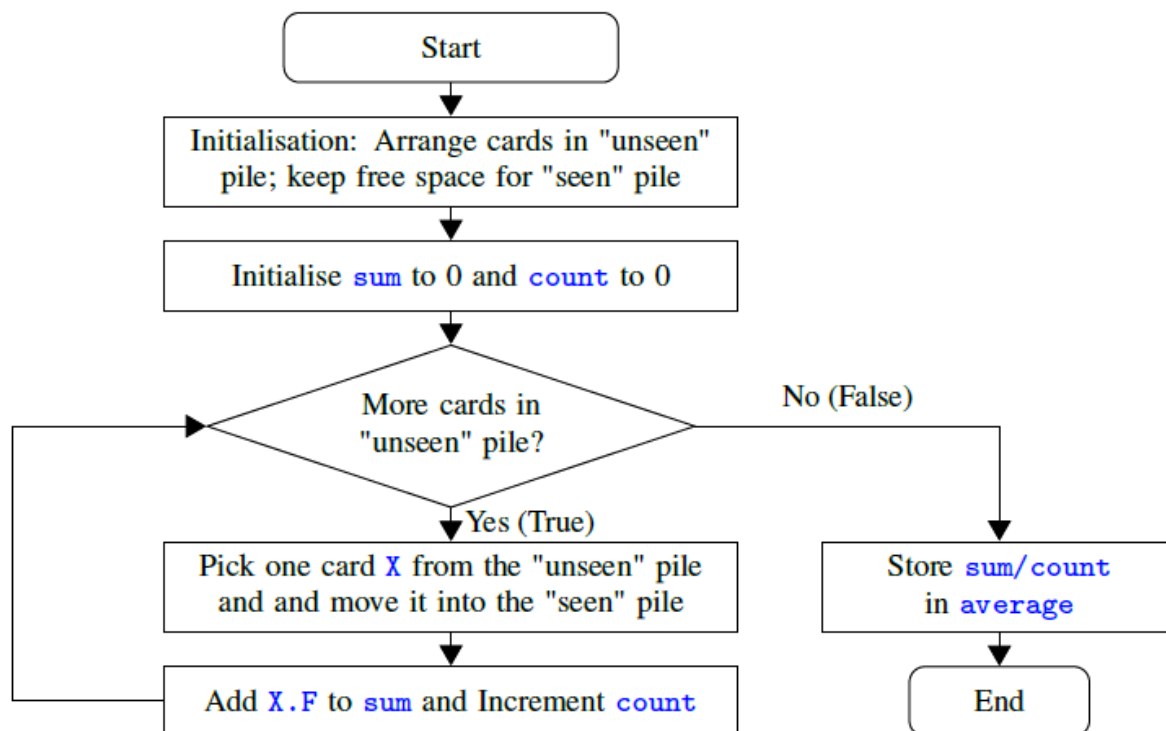


Figure 3.5: Generic flowchart for finding average

Finding the Average of a Field (One-Pass Method)

1. Problem

We have a set of records (cards). Each card has a field value, such as physics marks, bill amount, or letter count. We need to find the average of that field.

Average Formula

$$\text{Average} = \frac{\text{Total value (sum)}}{\text{Number of cards (count)}}$$

2. Example Data (Cards)

Card 1 Name: Anil
Physics Marks: 70

Card 2 Name: Binu
Physics Marks: 80

Card 3 Name: Chitra
Physics Marks: 90

... (more cards possible)

3. One-Pass Idea

- 1 Start with sum = 0 and count = 0.
- 2 Take one card from the unseen pile.
- 3 Move it to the seen pile.
- 4 Add the field value to sum.
- 5 Increase count by 1.
- 6 Repeat until no cards are left.
- 7 Finally, calculate average = sum / count.

4. Flowchart

```

graph TD
    Start([Start]) --> Init[sum = 0  
count = 0]
    Init --> Decision{Are there cards in the unseen pile?}
    Decision -- No --> AvgCalc[average = sum / count]
    AvgCalc --> End([End])
    Decision -- Yes --> SelectCard[Select a card X from unseen pile]
    SelectCard --> MoveCard[Move X to seen pile]
    MoveCard --> SumAdd[sum = sum + X.F]
    SumAdd --> CountInc[count = count + 1]
    CountInc --> Decision
  
```

5. Step-by-Step Execution (One Pass)

Step	Card X	X.F (Physics Marks)	sum (after adding)	count (after adding)	Explanation
1	Card 1 (Anil)	70	0 + 70 = 70	0 + 1 = 1	Add 70, count becomes 1
2	Card 2 (Binu)	80	70 + 80 = 150	1 + 1 = 2	Add 80, count becomes 2
3	Card 3 (Chitra)	90	150 + 90 = 240	2 + 1 = 3	Add 90, count becomes 3
After all cards are processed			sum = 240	count = 3	Average = 240 / 3 = 80

6. Result

Average = sum / count = 240 / 3 = 80

Thus, the average physics mark is **80**.

7. Two-Pass vs One-Pass

Two-Pass Method	One-Pass Method
1. First pass: calculate sum 2. Second pass: calculate count 3. average = sum / count	1. Calculate sum and count together in one pass. 2. average = sum / count
Inefficient (cards examined twice)	Efficient (cards examined only once)

8. Pseudocode

```

sum = 0
count = 0
while there are cards in the unseen pile:
    choose a card X
    move X to the seen pile
    sum = sum + X.F
    count = count + 1
average = sum / count
  
```

Accumulator

An **accumulator** is a variable that stores a result while we process data one item at a time.

Simple examples

- sum accumulates values by **adding** them.
- marksList accumulates values by **appending** them to a list.
- A product variable can accumulate values by **multiplying** them.

The accumulator must first be given an appropriate empty value:

```
sum = 0
```

```
marksList = []
```

```
product = 1
```

We use 0 for addition, [] for collecting items, and 1 for multiplication because these values do not change the result when the operation begins.

General pattern

An accumulator usually follows this process:

text

Read one data item



Process the item



Add the processed result to the accumulator



Repeat for all items

The two stages are often called:

- **Map:** Process each item and produce a useful value.
- **Reduce:** Combine that value with the accumulator.

text

Map → Reduce → Map → Reduce → Map → Reduce

Example: total packed items

Suppose we want to find the total number of packed items purchased by all customers.

A shopping bill may contain rows such as:

text

Bill 1: Rice, Milk, Shirts → 3 item rows

Bill 2: Tomatoes, Soap → 2 item rows

Bill 3: Books, Pens, Bags, Shoes → 4 item rows

We should not add quantities such as 1.5 kg tomatoes, 0.5 litres milk, and 2 shirts, because they use different units. Instead, we count each bill row as one packed item.

Step 1: Map

For each bill, count the number of item rows:

text

Bill 1 → 3

Bill 2 → 2

Bill 3 → 4

Step 2: Reduce

Add each result to the accumulator:

text

total = 0

total = 0 + 3 = 3

total = 3 + 2 = 5

total = 5 + 4 = 9

Therefore:

text

Total packed items = 9

General pseudocode

text

total = 0

for each bill:

 packageCount = number of item rows in the bill

 total = total + packageCount

display total

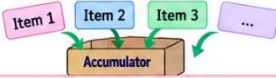
Here:

- packageCount is the value produced by the **map** operation.
- total is the accumulator.
- total = total + packageCount is the **reduce** operation.

In short, an accumulator stores a running result while data is processed, and the processing can happen before the result is added to the accumulator.

1. What is an Accumulator?

An **accumulator** is a variable that stores a result while we process data one item at a time.



We keep updating the accumulator as we process each item.

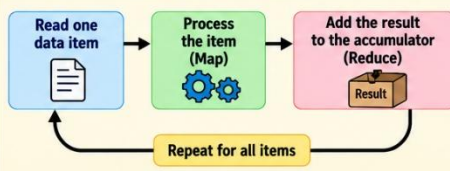
2. Simple Examples

Purpose	Accumulator	How it accumulates
Find sum	<code>sum = 0</code>	Add each value to sum
Collect values in a list	<code>marksList = []</code>	Append each value to the list
Find product	<code>product = 1</code>	Multiply each value with product

3. Why these initial values?

Operation	Initial value	Reason
Addition (sum)	0	0 does not change the sum.
Collecting items (list)	[]	Empty list can accept new items.
Multiplication (product)	1	1 does not change the product.

4. General Pattern of an Accumulator



5. Map and Reduce

Map	Reduce
Process each item and produce a useful value. Example: Count the number of item rows in each bill.	Combine that value with the accumulator. Example: Add the count to the total.

Map → Reduce → Map → Reduce → Map → Reduce

6. Real-life Example: Total Packed Items

A shopping bill contains rows of packed items. We want to find the total number of packed items purchased by all customers.

Bill	Items in the bill	Number of item rows
Bill 1	Rice, Milk, Shirts	3
Bill 2	Tomatoes, Soap	2
Bill 3	Books, Pens, Bags, Shoes	4

We count each bill row as one packed item. We do NOT add quantities like kg, litres, or units, because they use different measurement units.

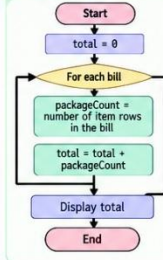
7. Step-by-Step Execution

Step	Bill	Map (packageCount)	Reduce (total = total + packageCount)	Total (after adding)
1	Bill 1 (Rice, Milk, Shirts)	3	0 + 3	3
2	Bill 2 (Tomatoes, Soap)	2	3 + 2	5
3	Bill 3 (Books, Pens, Bags, Shoes)	4	5 + 4	9

Final Result

Total packed items = 9

8. Flowchart



9. Pseudocode

```
total = 0
for each bill:
    packageCount = number
    of item rows in the bill
    total = total + packageCount
display total
```

- **packageCount** is the value produced by the **map** operation.
- **total** is the accumulator.
- **total = total + packageCount** is the **reduce** operation.

10. Key Takeaways

- ✓ An **accumulator** stores a running result while data is processed.
- ✓ It is updated one item at a time.
- ✓ Choose the correct initial value (0, [], or 1).
- ✓ Processing can happen before adding to the accumulator (**Map → Reduce**).
- ✓ Useful for sums, counts, lists, products, and many real-world problems.